Migrating Data from SQL to NoSQL with DLoader

Chimaroke Amadi · GoMyCode MSC checkpoint · 7 November 2026

To understand the topic properly, I did more than research it. I **built a small DLoader-style pipeline** and used it to migrate a real MySQL 9.7.4 database (108,520 rows) into MongoDB 8.2.6. I then **checked** that nothing was lost and measured what makes it fast or slow. Every number in this report comes from that run (`dloader/run.js`).

What I could and couldn't verify about DLoader. DLoader is not a commercial product. It comes from a research paper: *"DLoader: Migration of Data from SQL to NoSQL Databases"* by K. Rajaram, P. Sharma and S. Selvakumar, published in the Springer *Proceedings of the International Conference on Cognitive and Intelligent Computing* (2023, pp. 193–204). The full text is behind a paywall, and I found no public download or source code for the tool. So in this report, statements marked **(paper)** come from what is publicly known about the paper. Everything else is general migration practice, which I tested with my own implementation of the same three stages.

1. Introduction to Data Migration

What is data migration, and why is it important?

Data migration means moving data from one storage system to another, here from a relational (SQL) database to a NoSQL database. It usually also involves changing the *shape* of the data, because the two systems organize it differently. Companies migrate when their needs change: more traffic than one server can handle, data that no longer fits a fixed schema, the need to spread data across regions, or lower costs.

It matters because data is often a company's most valuable asset, and a migration is one of the riskiest moments in its life. A bad migration can **lose records, corrupt values, break relationships or cause downtime**, and it might not be noticed until much later. A good migration is planned, repeatable and verified.

Key differences between SQL and NoSQL databases

	SQL (relational: MySQL, PostgreSQL...)	NoSQL (MongoDB, Cassandra, Redis...)
Data model	Tables of rows and columns	Documents (JSON-like), key-value pairs, wide columns or graphs
Schema	Fixed and declared up front (`CREATE TABLE`)	Flexible: documents in one collection can have different fields
Relationships	Foreign keys + `JOIN` at query time	Usually **embedded** in the document or duplicated, and joins are limited or expensive
Design starts from...	The data itself (normalize to avoid repetition)	The **queries** the application will run
Scaling	Mostly vertical (a bigger server)	Built to scale horizontally (more servers, sharding)
Consistency	Strong ACID transactions	Often eventual or tunable consistency. MongoDB supports multi-document ACID transactions, but they cost more
Query language	Standard SQL	Specific to each database (MongoDB query API, CQL...)

2. Overview of DLoader

What is DLoader, and what role does it play?

DLoader is a **migration framework for moving data from a MySQL database into NoSQL databases**, specifically **MongoDB** (document store) and **Cassandra** (wide-column store) **(paper)**. It plays the role of the **ETL tool** in the migration: it Extracts the data from the source, Transforms it into the target's model, and Loads it. That way the migration is done by a repeatable program and not by hand.

Main features and capabilities

- **Extraction** of the data from a MySQL database **(paper)**.
- **Preprocessing**: applying transformations and standardizations to the data before it is loaded **(paper)**.
- **Column mapping**: mapping the MySQL table columns to fields in the NoSQL database **(paper)**.
- **Two kinds of target**: a document database (MongoDB) and a wide-column database (Cassandra), which lets the authors compare them. They report that in their experiments, Cassandra needed less storage space and performed better than the transformed MongoDB database **(paper)**.

To actually *work*, a tool built on these stages also needs the capabilities below. I included all of them in my implementation, and the rest of this report tests them: reading in batches, a consistent snapshot of the source, idempotent loading (safe to re-run), indexes, and validation.

3. The Migration Process

Steps to migrate from SQL to NoSQL with DLoader

1. **Assess the source.** List the tables, row counts, keys, relationships and data quality problems, and find out *how the application reads the data*.
2. **Design the target model.** Decide for each table whether it becomes its own collection, gets **embedded** in another document, or is **referenced** by id. This is the most important step, because a

NoSQL model is designed around the queries.

3. **Configure the mapping.** Write the column → field rules, the type conversions and the cleaning rules (DLoader's mapping and preprocessing stages).
4. **Extract** the rows from MySQL in batches, from one consistent snapshot.
5. **Transform** each batch: clean values, convert types, and combine related rows into documents.
6. **Load** the documents into the NoSQL database in bulk, then create the indexes.
7. **Validate**: compare counts, checksums and samples between source and target.
8. **Cut over**: switch the application to the new database, keep the old one read-only for a while, and have a rollback plan.

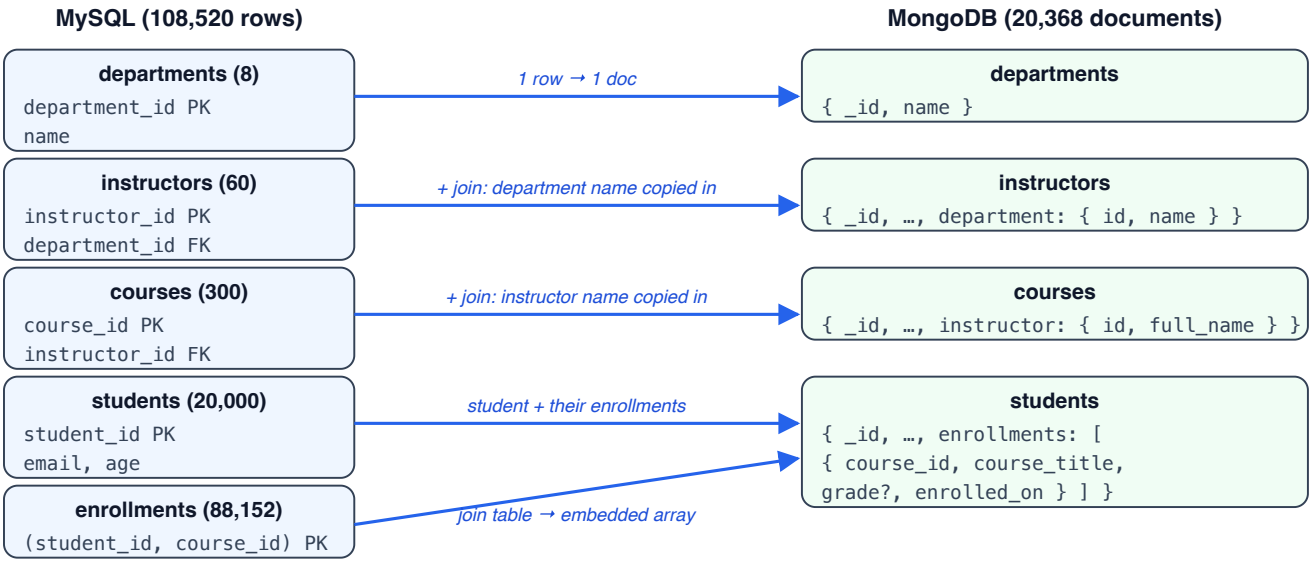
Common challenges and how to address them

Challenge	How to address it
Different data models: SQL relies on joins, which NoSQL doesn't do well	Redesign around the queries: embed data that is read together, reference data that is large or shared (section 4)
Data types that don't exist on the other side (DECIMAL, TINYINT(1) booleans, DATE, ENUM)	An explicit type-mapping table in the transform step, tested on real data
Dirty data: spaces, inconsistent case, invalid values	Standardize during preprocessing (my pipeline trims and lowercases emails, for example)
The source keeps changing during the migration	Read from a consistent snapshot, then catch up the changes (or briefly freeze writes, or use change-data-capture)
Lost integrity: no foreign keys in NoSQL	Check references during validation, and enforce rules in the application or with schema validation
Large volumes and slow migrations	Batching, bulk writes, indexes after the load, parallel workers (section 5)
Failures halfway through	Make loading **idempotent** (upsert by the original key), so the migration can simply be re-run
Application changes: every query has to be rewritten	Plan it as part of the project, and run old and new side by side before switching

4. Data Transformation

How DLoader handles transformation

DLoader handles it in its **preprocessing** stage, where it applies transformations and standardizations, and its **mapping** stage, where MySQL columns become NoSQL fields **(paper)**. In practice the transformation does three jobs: **clean** the values, **convert** the types, and **restructure** rows from several tables into documents. Here is the mapping I designed for the university database:



Example: one student before and after

MySQL: 1 row in `students`

student_id	full_name	email	age
50	"Ibrahim Danjuma"	" Student50@Student.UNI.edu "	18

+ 3 rows in `enrollments`

student_id	course_id	grade	enrolled_on
50	104	C	2026-09-03
50	145	F	2026-09-08
50	228	D	2026-09-08

```
// MongoDB: 1 document in "students"
{
  _id: 50,
  full_name: "Ibrahim Danjuma",
  email: "student50@student.uni.edu",
  age: 18,
  enrollments: [
    {
      course_id: 104,
      course_title: "Linear Algebra 110",
      grade: "C",
      enrolled_on: ISODate("2026-09-03")
    },
    {
      course_id: 145,
      course_title: "Networks 114",
      grade: "F",
      enrolled_on: ISODate("2026-09-08")
    },
    {
      course_id: 228,
      course_title: "Microeconomics 122",
      grade: "D",
      enrolled_on: ISODate("2026-09-08")
    }
  ],
  enrollment_count: 3
}
```

Look at what changed: the **3 enrollment rows were embedded** inside the student, along with the course title, so showing a student's courses needs no join. The messy email `" Student50@Student.UNI.edu "` was **trimmed and lowercased**, the dates became real **Date** values, and the **NULL grade was left out** of the document instead of being stored as `null`.

Other common transformations

In SQL	In NoSQL (MongoDB)	Why
Join table (`enrollments`)	Array embedded in the parent document	Read together. Fine as long as the array stays small (bounded)
Foreign key to a small lookup table	Copy the needed fields in (`department: { id, name }`)	Avoids a join on every read. Costs an update in several places if the name changes
Foreign key to a big or shared entity	Keep a reference (`course_id`)	Embedding a whole course in every student would repeat it thousands of times
`INT AUTO_INCREMENT` id	Kept as `_id` (or a new ObjectId + `legacy_id`)	Keeping it makes re-runs idempotent and lets you trace each document back to its row
`DECIMAL(10,2)` money	`Decimal128`	A JavaScript number (double) can't store 0.10 exactly
`TINYINT(1)`	`true` / `false`	MySQL has no real boolean type
`DATE` / `DATETIME`	BSON `Date` (UTC)	Allows date queries and sorting. Watch out for time zones
`NULL`	Field left out (or `null`)	Pick one rule and use it everywhere, so queries are predictable
For Cassandra: one normalized table	Several tables, one per query	Cassandra has no joins at all, so data is duplicated per access pattern

5. Performance Considerations

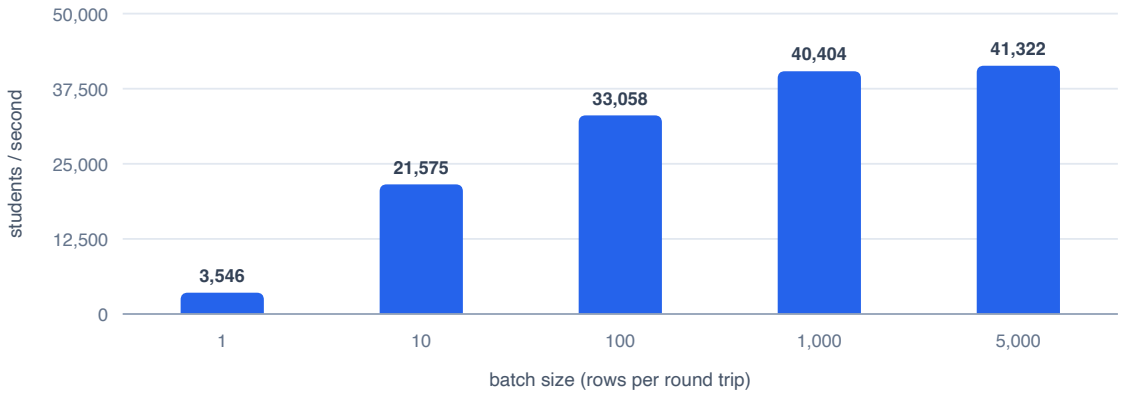
What to consider

- **Network round trips**: sending rows one by one is slow, so batch them.
- **How the source is read**: avoid `OFFSET` pagination and one query per row (the "N+1" problem), and make sure the source uses its indexes.
- **Write cost on the target**: bulk writes, write concern, and indexes being updated during the load.
- **Load on the production source**: a migration shouldn't slow down the live application, so throttle it or read from a replica.
- **Document size**: embedding too much makes documents large (MongoDB's limit is 16 MB per document).
- **Storage**: documents repeat field names and duplicated data (here, course titles inside every enrollment). In this run, the MySQL data took 6.1 MB. The same data in MongoDB was 10.2 MB uncompressed, but only 2.6 MB on disk, because MongoDB compresses its storage. So duplication costs space, but compression can more than make up for it.

How a DLoader-style pipeline optimizes it, measured

The whole migration (108,520 rows → 20,368 documents, plus indexes) took **736 ms** on my laptop. Here are the experiments behind that:

1. Batching. I loaded the 20,000 students with different batch sizes (median of 3 runs each):



Batch size	1	10	100	1,000	5,000
Students per second	3,546	21,575	33,058	40,404	41,322
3 runs (ms)	574 / 519 / 564	919 / 948 / 927	558 / 633 / 605	495 / 486 / 503	514 / 453 / 484

Going from 1 row per round trip to 5,000 made it **about 12× faster**. Above about 1,000 it stops improving, because the batches are then big enough that the round trips no longer matter. (The batch size 1 run was measured on 2,000 students, because it is so slow.)

2. The source database's statistics. My first benchmark gave strange results. To find the cause, I timed each stage separately and found that the query reading each batch's enrollments was slow. MySQL's statistics were out of date after the bulk insert, so it chose a bad plan: it scanned *every course* and its enrollments, instead of jumping straight to the batch's students through the primary key. Running `ANALYZE TABLE` fixed it:

Query for 1,000 students	Time	MySQL's plan
Before `ANALYZE TABLE`	15 ms	Covering index scan on c using uq_courses_title → Index lookup on e using idx_enrollments_course (course_id = c.course_id)
After `ANALYZE TABLE`	4.9 ms	Index range scan on e using PRIMARY over student_id IN (...) → Single-row index lookup on c using PRIMARY (course_id = e.course_id)

That's 3.1× faster for 1,000 ids (and 6.8 → 0.6 ms for 100 ids). My pipeline now refreshes the statistics before extracting. The lesson: **a migration is only as fast as its slowest query, so check the source's query plans** with `EXPLAIN`.

3. Other techniques in the pipeline:

- **Keyset pagination** (`WHERE id > last ORDER BY id LIMIT n`) instead of `OFFSET`, which gets slower and slower on big tables.
- **One query per batch** for the enrollments (`WHERE student_id IN (…)`), not one per student: 20 queries instead of 20,000.
- **Unordered bulk writes** (`bulkWrite`, `ordered: false`), so MongoDB can apply each batch efficiently.
- **Indexes after the load.** The usual advice is to build indexes once at the end, not update them on every insert. At this size I measured a **small advantage** for creating them after the load (median 766 ms with the indexes created first, 631 ms after the load), and the difference changed from run to run. At 20,000 documents it hardly matters. The advice is for loads of millions of documents.

6. Consistency and Integrity

How the pipeline keeps the data consistent

- **One consistent snapshot.** All reads happen inside `START TRANSACTION WITH CONSISTENT SNAPSHOT, READ ONLY`, so every batch sees the database as it was at the same moment, even if the application keeps writing. Without it, a student could be copied before a new enrollment and the enrollment missed.
- **Idempotent loading.** Each document is *upserted* using its SQL primary key as `_id`. Running the migration twice gives the same result with no duplicates, so a failure halfway through is fixed by simply running it again.
- **Integrity rules move into the code.** MongoDB has no foreign keys, so the pipeline checks references during validation instead.
- DLoader's **standardization** step **(paper)** also helps consistency: the same value is always written the same way (for example, lowercase emails).

Strategies to verify the migrated data, and my results

Check	MySQL	MongoDB	Result
count: departments	8	8	✔ match
count: instructors	60	60	✔ match
count: courses	300	300	✔ match
count: students	20,000	20,000	✔ match
count: enrollments (rows vs embedded array items)	88,152	88,152	✔ match
checksum: sum of all ages	546,894	546,894	✔ match
checksum: number of each grade (A–F, none)	A: 12,308, B: 12,260, C: 13,314, D: 11,072, E: 12,348, F: 12,939, none: 13,911	A: 12,308, B: 12,260, C: 13,314, D: 11,072, E: 12,348, F: 12,939, none: 13,911	✔ match
checksum: total credits	1,031	1,031	✔ match
integrity: enrollments pointing to a missing course	0	0	✔ match
sample: 200 random students compared field by field	0	0	✔ match

The checks go from cheap to thorough:

- **Counts** catch lost or duplicated records.
- **Checksums** (sums and group counts calculated independently on each side) catch wrong values.
- The **integrity** check catches broken references.
- The **record-by-record comparison** rebuilds random students from SQL and compares every field.

Does the validation actually catch problems? I broke the target on purpose in 3 ways: student 123: last enrollment removed; student 456: email changed; student 789: document deleted. The validation **failed, as it should**, in 5 different checks:

- count: students: MySQL 20,000, MongoDB 19,999
- count: enrollments (rows vs embedded array items): MySQL 88,152, MongoDB 88,146
- checksum: sum of all ages: MySQL 546,894, MongoDB 546,872

- checksum: number of each grade (A–F, none): MySQL A: 12,308, B: 12,260, C: 13,314, D: 11,072, E: 12,348, F: 12,939, none: 13,911, MongoDB A: 12,307, B: 12,260, C: 13,314, D: 11,071, E: 12,348, F: 12,937, none: 13,909
- sample: 20,000 random students compared field by field: MySQL 0, MongoDB 3 (different: 456, 789, 123)

Then I simply **re-ran the migration**. Because it is idempotent, it repaired all three problems, and every check passed again with exactly 20,000 students (no duplicates).

7. Practical Application: A Migration Plan

The scenario: a university runs its student system on MySQL (the schema from my previous SQL checkpoint): departments, instructors, courses, students and enrollments. The student portal is slow because every "my courses" page joins three tables, and the university wants to move the portal to MongoDB.

The plan

Phase	What happens	Done when
1. Assess	Count rows, list keys and relationships, profile data quality (messy emails, NULL grades), list the portal's queries	We know what to move and how it's read
2. Design	Target model as in section 4: students embed their enrollments, courses embed their instructor's name, departments stay small	Every portal query can be answered by 1 document read
3. Configure DLoader	Column → field mapping, type conversions, cleaning rules, batch size 1,000, indexes to create	The mapping is written in code and reviewed
4. Rehearse	Run the full migration on a **copy** of production and run all the validation checks	All checks pass, the time is measured, and a re-run causes no duplicates
5. Migrate	Short write freeze (or snapshot + catch-up), run DLoader, validate	All checks pass on the real data
6. Cut over	Switch the portal to MongoDB; MySQL stays read-only as a fallback	The portal works, and errors and response times are normal
7. Monitor and clean up	Compare a sample of answers between old and new for a few weeks, then retire MySQL	No differences found

Rollback plan: until MySQL is retired, switching the portal back is just a configuration change, because MySQL was never modified.

How I implemented it

I implemented phases 2 to 5 and ran them on a temporary copy filled with 20,000 generated students:

```
npm run migrate
```

1. `run.js` creates the MySQL schema and the test data (108,520 rows, including a few messy emails on purpose).
2. `pipeline.js` refreshes the statistics, opens a consistent snapshot, then for each table: **extracts** 1,000 rows at a time, **transforms** them (clean, convert, embed) and **loads** them with bulk upserts. Then it creates the indexes.

3. `validate.js` runs the 10 checks above. All of them passed.
4. The rehearsal tests are included too: breaking data on purpose, re-running to repair it, and measuring performance.

The same question, *"show student 50 with their courses"*, needs a 3-table `JOIN` returning 3 rows in MySQL, but **a single** `findOne` returning 1 document in MongoDB. That's exactly the access pattern the new model was designed for.

8. Case Studies

Netflix: Oracle → SimpleDB → Cassandra

When Netflix moved to the cloud around 2010–2011, it moved its data out of its own Oracle databases, first to Amazon SimpleDB and S3, and later to Cassandra. Talks from the time describe the method: **dual writes** to both systems, a bulk **"forklift"** of the existing data, and a **consistency checker** to compare the two. Its billing data, spread over many Oracle tables, was transformed into a much simpler Cassandra structure. The main motivations were availability and scale: the central Oracle database was a single point of failure.

Craigslist: MySQL archive → MongoDB

Craigslist kept its live postings in MySQL but moved its **archive** of old postings to MongoDB (2011, over two billion documents according to MongoDB's case study). Every schema change on the huge archive took a very long time, and archive data piled up in the production database. Each expired posting became one JSON-like document. MySQL stayed for the live site, so this was a **partial** migration of the data that suited documents best.

Diaspora: MongoDB → MySQL (the other direction)

The Diaspora social network started on MongoDB and, after months in production, **moved to MySQL**. Sarah Mei, one of its developers, explained that social data (users, friends, posts, comments) is deeply **relational**. In documents it had to be duplicated and kept in sync by hand, which caused inconsistencies.

Lessons learned

- **Choose by access pattern, not fashion.** Craigslist's archive (write once, read as a whole) fitted documents perfectly. Diaspora's interconnected social data did not.
- **You don't have to migrate everything.** Craigslist kept MySQL for the live site, and a hybrid is often the best answer.
- **Plan for running both systems in parallel**, with dual writes and a consistency checker (Netflix), the same idea as my validation step.
- **The data model must be redesigned**, not copied table by table.

9. Conclusion

Advantages of moving to NoSQL
Horizontal scaling across many servers
Flexible schema: new fields without a migration
Fast reads when a document holds everything a page needs (1 read instead of a 3-table join)
Natural fit for JSON APIs and semi-structured data
High availability through replication (Netflix's main reason)

Disadvantages and risks
Joins and ad-hoc queries become harder
Duplicated data has to be kept in sync by the application
Weaker built-in integrity (no foreign keys), and transactions are more limited or costly
Every query in the application has to be rewritten
Often more storage, and the migration itself is a risky project

When I would recommend DLoader. I'd use the DLoader approach (extract → preprocess/standardize → map → load, plus validation) for a **planned, batch migration of MySQL data into MongoDB or Cassandra**, when:

- the data is mostly **read as a whole** (like a student with their courses, or an archived posting);
- the access patterns are known, so the target model can be designed well;
- a short write freeze or a catch-up step is acceptable.

I would **not** use it when:

- the data is highly relational and transactional (bank transfers, social graphs, like Diaspora);
- both databases must stay in sync continuously for a long time. That calls for change-data-capture or replication tools.

Also, because DLoader is a research prototype with no public release that I could find, a real company would probably rely on a maintained tool built on the same ideas. Its value for me was showing clearly **what every migration needs**: careful mapping, cleaning, batching and validation.

Sources

- K. Rajaram, P. Sharma, S. Selvakumar, "DLoader: Migration of Data from SQL to NoSQL Databases", *Proc. Int. Conf. on Cognitive and Intelligent Computing*, Springer (Cognitive Science and Technology), 2023, pp. 193–204. https://link.springer.com/chapter/10.1007/978-981-19-2358-6_19 (metadata confirmed via Crossref)
- Netflix: S. Anand, "NoSQL @ Netflix", QCon London 2011, https://www.infoq.com/presentations/NoSQL-Netflix · A. Cockcroft, "Migrating Netflix from Oracle to Global Cassandra" (2011), https://www.slideshare.net/slideshow/migrating-netflix-from-oracle-to-global-cassandra/8600029
- Craigslist: MongoDB case study, https://www.mongodb.com/resources/solutions/use-cases/mongodb-case-study-craigslist · TheServerSide, https://www.theserverside.com/feature/How-NoSQL-MySQL-and-MogoDB-worked-together-to-solve-a-big-data-problem
- Diaspora: S. Mei, "Why You Should Never Use MongoDB" (2013), http://www.sarahmei.com/blog/2013/11/11/why-you-should-never-use-mongodb/ · "MongoDB to MySQL: The How and the Why", RubyConf 2011, https://www.rubyevents.org/talks/mongodb-to-mysql-the-how-and-the-why

---

Measured on MySQL 9.7.4 and MongoDB 8.2.6 (Node v20.19.4), temporary local servers, one laptop. Timings vary between machines; the comparisons between them are what matter. Reproduce with `npm run migrate && npm run report`.